SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Sasikumar Megha	**Reg. No.:**	192471038
Section / Batch:	3rd CSE B's	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

92

Code of Conduct

I Sasikumar Megha (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Megha

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	✓			
Application of Engineering Concepts	25%	✓			
Solution Design	25%	✓			
Use of Tools	15%	✓			
Analysis	10%		✓		
Professionalism	5%	✓			
Total Marks (100):		92			

Faculty In-charge	Course Coordinator	Head of Department
Name: Rajeswari	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: 31/8/26	Date: _____	Date: _____

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06

COURSE OUTCOME COVERED

CO 4: Solve optimization problems using dynamic programming and greedy strategies.

(BL4, BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 7

Total Marks: 35

Annexure A

Industry Problem Solving Task

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Industry Problem Solving Task

Q90. Smart Retail Supply Chain Optimization — Dynamic Programming

1. Analysis

Dynamic Programming (DP) can optimize supply chain decisions by dividing the problem into smaller stages such as **inventory management, warehouse selection, and product distribution**.

The system can define:

- **State:** Inventory available at each warehouse and time period.
- **Decision:** Quantity to order, store, or distribute.
- **Cost:** Ordering + storage + transportation + shortage cost.
- **Goal:** Minimize total supply-chain cost.

A recurrence can be represented as:

$$DP[t][i] = \min(DP[t-1][j] + Cost(j, i))$$

where t represents the time period and i represents the current inventory/distribution state.

2. Constraints

- Demand fluctuations
- Limited warehouse capacity
- Transportation costs
- Delivery deadlines
- Supplier availability
- Inventory holding costs

3. Optimized Model

Input → Demand Forecast → Inventory State → DP Optimization → Distribution Decision → Delivery

The DP system calculates the lowest-cost combination of ordering and transportation decisions.

4. Scalability

For global operations, the problem can become very large because there are many warehouses, products, suppliers, and countries.

Scalability can be improved using:

- State-space reduction

- Regional optimization
- Parallel processing
- Demand forecasting
- Hierarchical DP

5. Performance Metrics

Metric	**Purpose**
Total Cost	Measures optimization
Delivery Time	Measures speed
Inventory Level	Measures stock efficiency
Stock-out Rate	Measures product availability
Transportation Cost	Measures logistics efficiency

6. Interpretation

A successful DP model reduces unnecessary inventory and transportation costs while maintaining product availability. This improves **profitability, delivery reliability, and customer satisfaction**.

Conclusion: Dynamic Programming is suitable because it considers multiple decisions over time and finds a cost-effective supply-chain strategy.

Q91. Intelligent Traffic Flow Prediction Optimization — Data Analysis + Graph Models 1.

Analysis

Traffic networks can be represented using a **graph**:

- **Vertices:** Junctions/traffic signals
- **Edges:** Roads
- **Edge weight:** Travel time, traffic density, or distance Historical

traffic data can be analyzed to predict future congestion.

The system can use:

Traffic Data → Data Cleaning → Traffic Prediction → Graph Analysis → Route/Signal Optimization

2. Constraints

- Changing traffic conditions
- Accidents
- Weather

- Road capacity
- Peak-hour traffic
- Incomplete or noisy sensor data

3. Predictive Traffic Management System

1. Collect traffic data from sensors and GPS.
2. Remove incorrect or missing data.
3. Calculate traffic density and average speed.
4. Predict future traffic levels.
5. Assign predicted values as graph edge weights.
6. Use shortest-path algorithms to identify efficient routes.
7. Adjust traffic signals based on congestion.

For example, **Dijkstra's algorithm** can find the minimum-time route when road weights are known.

4. Scalability

Large cities may contain thousands of roads and junctions. Scalability can be achieved using:

- Graph partitioning
- Parallel processing
- Real-time data streams
- Hierarchical road networks
- Distributed computing

5. Performance Metrics

- Prediction accuracy
- Average travel time
- Waiting time
- Number of congested roads
- Algorithm execution time
- CPU/memory usage

6. Interpretation

If predicted traffic is accurate and average travel time decreases, the system successfully improves urban mobility. Reduced congestion also results in lower fuel consumption and pollution.

Conclusion: Combining data analysis with graph algorithms enables intelligent, real-time traffic management.

Q92. Smart Energy Storage Management — Greedy + Dynamic Programming

1. Analysis

Energy providers need to decide **when to charge, store, and discharge batteries**.

Two approaches can be used:

Greedy Algorithm: Select the locally best decision, such as charging when electricity prices are low.

Dynamic Programming: Consider multiple future time periods to find the globally optimal charging/discharging schedule.

Example:

$$DP[t][b] = \text{minimum cost at time } t \text{ with battery level } b$$

2. Constraints

- Battery capacity
- Charging/discharging rate
- Charging cycles
- Electricity price changes
- Peak demand
- Battery degradation
- Renewable energy availability

3. System Design

Energy Forecast → Price Forecast → Battery State → Greedy/DP Optimization → Charge/Discharge Decision

Greedy can be used for quick real-time decisions, while DP can be used for more accurate scheduling.

4. Scalability

For large energy networks:

- Optimize batteries independently when possible.
- Divide the network into regions.
- Use parallel computation.
- Reduce unnecessary DP states.
- Use rolling time windows.

5. Performance Metrics

Metric	**Evaluation**
Energy Cost	Lower is better
Peak Demand	Lower is better

Battery Utilization	Higher is better
Computational Time	Lower is better
Battery Cycles	Lower excessive cycling is better

6. Interpretation

The optimized system stores energy during low-cost periods and releases it during peak demand. This reduces electricity costs and improves grid stability.

Conclusion: Greedy provides fast decisions, while DP provides better long-term optimization. A hybrid approach is effective for real-time energy management.

Q93. Online Marketplace Order Matching — Search + Graph Algorithms

1. Analysis

An e-commerce marketplace must match **buyers with suitable sellers**.

Search algorithms can quickly find sellers based on:

- Product
- Price
- Location
- Availability
- Rating

Graph algorithms can represent buyers and sellers as nodes, with edges representing possible transactions.

Example:

Buyer → Product → Seller

Edge weights can represent price, delivery time, or matching score.

2. Constraints

- Product availability
- Price
- Seller location
- Delivery time
- Seller rating
- Large number of users
- Rapidly changing inventory

3. Matching System

1. Buyer submits an order.
2. Search index finds available sellers.
3. Matching graph is created.
4. Each buyer-seller edge receives a score.
5. Best seller is selected.
6. Inventory is updated.

A weighted matching algorithm can maximize overall transaction efficiency.

4. Scalability

Millions of buyers and sellers can create a very large matching graph.

Scalability can be improved using:

- Search indexing
- Database partitioning
- Caching
- Graph partitioning
- Parallel processing
- Approximate matching

5. Performance Metrics

- Search time
- Matching time
- Match accuracy
- Order completion rate
- Transaction throughput
- Server resource usage

6. Interpretation

A fast and accurate matching system reduces order processing time and improves customer satisfaction. Sellers also receive more relevant orders.

Conclusion: Search provides fast candidate retrieval, while graph algorithms enable optimized buyerseller matching.

Q94. Smart Water Resource Allocation — Flow Algorithms

1. Analysis

Water distribution can be modeled as a **flow network**.

- **Source:** Reservoir/dam
- **Intermediate nodes:** Treatment plants and pipelines
- **Destination:** Cities/zones
- **Edge capacity:** Maximum pipeline capacity
- **Flow:** Quantity of water transported

A **Maximum Flow algorithm** can determine the maximum amount of water that can reach different zones.

2. Constraints

- Limited water supply
- Pipeline capacity
- Variable demand
- Leakage
- Seasonal changes
- Minimum required water levels

3. Allocation System

Water Source → Treatment Plant → Pipeline Network → Zones

Steps:

1. Collect supply and demand information.
2. Build the flow network.
3. Assign capacities to pipelines.
4. Assign demand requirements.
5. Run a flow algorithm.
6. Allocate water according to capacity and priority.
7. Monitor actual consumption.

Algorithms such as **Ford-Fulkerson** or **Edmonds-Karp** can be used.

4. Scalability

For large regions:

- Divide the network into zones.
- Process independent regions separately.
- Use efficient flow algorithms.
- Update only changed network sections.
- Use distributed processing.

5. Performance Metrics

- Water delivered
- Unmet demand
- Pipeline utilization
- Distribution efficiency
- Computational time
- Water loss

6. Interpretation

If the algorithm increases water delivered while reducing shortages and wastage, the allocation is efficient.

Conclusion: Flow algorithms provide a mathematical method for distributing limited water resources efficiently and sustainably.

Q95. Intelligent Document Classification — Search + Machine Learning

1. Analysis

Document classification automatically assigns documents to categories such as:

- Finance
- Education
- Legal
- Technical
- News

Machine Learning performs classification, while search algorithms help retrieve relevant documents and features.

The overall process is:

Documents → Preprocessing → Feature Extraction → ML Classification → Search/Indexing → Category

2. Constraints

- Very large datasets
- High-dimensional features
- Different document formats
- Noisy text
- Similar categories
- Feature extraction cost

- Changing document types

3. Classification System

1. Collect documents.
2. Remove unnecessary words and symbols.
3. Tokenize the text.
4. Convert text into numerical features using methods such as TF-IDF.
5. Train a classifier.
6. Classify new documents.
7. Store classified documents in a searchable index.
8. Retrieve documents quickly using search.

Possible classifiers include:

- Naive Bayes
- Decision Tree
- SVM
- Neural Networks

4. Scalability

For millions of documents:

- Distributed storage
- Parallel feature extraction
- Batch processing
- Search indexing
- Incremental model training
- Dimensionality reduction

5. Performance Metrics

Metric	**Meaning**
Accuracy	Overall correct classifications
Precision	Correct positive predictions
Recall	Correctly identified relevant documents
F1-score	Balance of precision and recall

Classification Time	Processing speed
Search Time	Retrieval speed

6. Interpretation

High accuracy and F1-score indicate that documents are being categorized correctly. Fast search reduces the time required to retrieve information.

Conclusion: Combining machine learning with search algorithms provides an efficient system for automatically organizing and retrieving large document collections.